Adaptive Hopfield Network: Rethinking Similarities in Associative Memory

Wang, Pan, Shen, Zhang, Wang & Li — ICLR 2026

Machine Learning in Practice Reading Group

Duke B&B

October 23, 2026

Presented by Finn Guo

The paper in one slide

Claim. Hopfield networks / attention retrieve memories with a *fixed* similarity (usually $\boldsymbol{\xi}^\top \mathbf{x}$). A fixed similarity cannot guarantee the *right* memory is retrieved. Learn the similarity instead.

Two ideas carry the whole paper:

1. **Retrieval is classification.** Ask "which stored pattern produced this query?" The ideal similarity is the log-likelihood $\log p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi})$.
2. **Sort, then weight.** Compare query and memory dimension by dimension, sort the per-dimension scores from best to worst, and learn how much to trust each rank (e.g. "ignore the worst few dimensions").

My verdict (details in Sec. 4 and 7).

- Idea 1 is clean and useful as a way of thinking.
- Idea 2 works empirically; the ablations support it.
- The theorems are weaker than the abstract suggests: they cover an idealized family, not the model that is trained.

Section 1: Introduction — the limitation

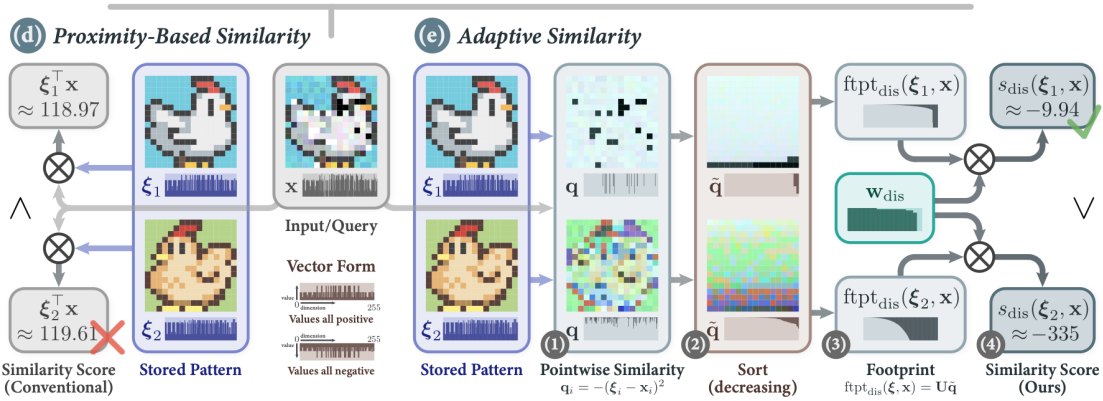
All modern Hopfield networks retrieve in three steps (Millidge et al. 2022):

$$\mathbf{y} = \mathcal{T}(\mathbf{x}) = \boldsymbol{\Xi}\, \mathrm{sep}\big(\mathrm{sim}(\boldsymbol{\Xi}, \mathbf{x})\big) \qquad \text{similarity} \;\rightarrow\; \text{separation} \;\rightarrow\; \text{read-out}$$

Model	$\mathrm{sim}(\boldsymbol{\xi}, \mathbf{x})$	$\mathrm{sep}(\mathbf{s})$
M-Hop (Ramsauer '21)	$\boldsymbol{\xi}^\top \mathbf{x}$	softmax
S-Hop (Hu '23)	$\boldsymbol{\xi}^\top \mathbf{x}$	sparsemax
U-Hop (Wu '24)	$\boldsymbol{\xi}^\top \boldsymbol{\Phi}^\top \boldsymbol{\Phi} \mathbf{x}$	α-entmax
A-Hop (this paper)	$\mathbf{w}^\top \mathrm{ftpt}(\boldsymbol{\xi}, \mathbf{x})$	softmax

Decades of work changed the **separation** step. The **similarity** step stayed a dot product. (Full list: Table 1 of the paper.)

Section 1: A concrete failure of the dot product



The query $\mathbf{x}$ is a noisy + masked copy of ξ_1. **(d)** The dot product scores ξ_2 higher (119.6 vs. 119.0) and retrieves the wrong pattern. **(e)** The adaptive similarity scores per pixel, sorts, accumulates, and weights: ξ_1 wins (-9.9 vs. -335).

Section 1: Intuition and potential applications

Intuition. A query is a *corrupted copy* of one stored pattern. If we knew how copies get corrupted, we would know how to compare them.

Applications explored by the authors: synthetic memory retrieval, MNIST retrieval, tabular classification (incl. a cardiovascular-disease dataset), image classification, multiple-instance learning.

Clinical data: the three corruption types have natural analogues

- **Noisy**: measurement noise in vitals and labs.
- **Masked**: tests never ordered, codes never entered — some dimensions are simply wrong or missing.
- **Biased**: systematic offsets between sites, assays, or coding practices.

Relevance to our work. Retrieval over a patient's stored history (e.g. concept-level memory with cosine top-L) uses exactly the kind of fixed similarity this paper questions.

Section 2: Background

Modern Hopfield network = attention (Ramsauer et al. 2021)

$$E(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^\top\mathbf{x} - \mathrm{lse}(\boldsymbol{\Xi}^\top\mathbf{x}), \qquad \mathcal{T}(\mathbf{x}) = \boldsymbol{\Xi}\,\mathrm{softmax}(\boldsymbol{\Xi}^\top\mathbf{x})$$

One update step lowers the energy; with keys $= \boldsymbol{\Xi}$ and values $= \boldsymbol{\Xi}$ this is one attention read.

How retrieval is usually judged: ϵ-retrieval (Def. 1)
Pattern $\boldsymbol{\xi}$ is ϵ-retrieved if $\|\mathbf{y} - \boldsymbol{\xi}\|_2 \leq \epsilon$. This says the output is *close to some memory* — not that it is the *right* memory.

Also needed: Bayes' rule and the Bayes-optimal classifier; the concave–convex procedure (CCCP) for Thm. 2.

Resources

- Ramsauer et al., *Hopfield Networks is All You Need*, ICLR 2021.
- Millidge et al., *Universal Hopfield Networks*, ICML 2022 (the sim/sep/read-out view).
- Krotov, Hoover, Ram & Pham, *Modern Methods in Associative Memory*, 2025 (tutorial review).

Section 3.1: Variant distribution and correct retrieval

Variant distribution $\mathcal{V}(\Xi)$ (Def. 2): a joint distribution over (stored pattern, query) pairs. Think of it as: pick $\boldsymbol{\xi}_k$, then corrupt it into $\mathbf{x}$.

Correct retrieval (Def. 3): the memory closest to the output is the most probable origin of the query,

$$\arg\min_{\boldsymbol{\xi}' \in \Xi} \|\mathbf{y} - \boldsymbol{\xi}'\|_2 \;=\; \arg\max_{\boldsymbol{\xi}' \in \Xi} p_{\mathcal{V}}(\boldsymbol{\xi}' \mid \mathbf{x}) \;=\; \arg\max_{\boldsymbol{\xi}' \in \Xi} p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi}')\, p_{\mathcal{V}}(\boldsymbol{\xi}') \quad \text{(Eq. 2)}$$

Reading it as classification (presenter's observation)

Retrieval over N memories is an N-class problem; correct retrieval is the Bayes decision rule. With a uniform prior and $\mathrm{sim}(\boldsymbol{\xi}_k, \mathbf{x}) = \log p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi}_k)$,

$$\mathrm{softmax}_k\big(\log p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi}_k)\big) = p_{\mathcal{V}}(\boldsymbol{\xi}_k \mid \mathbf{x}).$$

So the attention weights would be *exactly the posterior*. A dot product implicitly assumes $\log p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi}) \propto \boldsymbol{\xi}^\top \mathbf{x}$.

Note: the paper drops the prior ("typically uniform"); App. A.3.2 suggests a bias term $b_k = \log p_{\mathcal{V}}(\boldsymbol{\xi}_k)$ otherwise.

Section 3.1: Three canonical variants (Defs. 4–6)

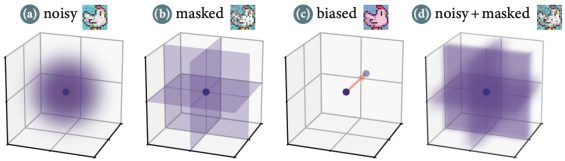


Figure 1: Visualization of probability density function $p_{\mathcal{V}}(\mathbf{x}|\boldsymbol{\xi})$ for noisy, masked, biased, and noisy + masked variants. Darker regions indicate larger $p_{\mathcal{V}}(\mathbf{x}|\boldsymbol{\xi})$, and the central dark point represents $\boldsymbol{\xi}$.

Fig. 1: density of $p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi})$ around one memory (dark dot) for each variant.

Variant	How $\mathbf{x}$ is made	Best similarity
Noisy	$\mathbf{x} = \boldsymbol{\xi} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathrm{diag}(\boldsymbol{\sigma}))$	$-\sum_i (x_i - \xi_i)^2 / \sigma_i$
Masked	each dim. replaced w.p. p_{masked} by a random value	# dims with $x_i = \xi_i$
Biased	$\mathbf{x} = \boldsymbol{\xi} + \mathbf{d}$	$-\lVert\mathbf{x} - \boldsymbol{\xi} - \mathbf{d}\rVert_2^2$

Takeaways

- None of the three is a dot product.
- The masked one is discrete: no continuous similarity is optimal for it (App. A.2.1).
- Real data mixes variants and $p_{\mathcal{V}}$ is unknown $\Rightarrow$ **learn** it from samples.

Section 3.2: Similarity footprint

Start from a **base similarity** that adds up over dimensions, e.g. $\mathrm{dis}(\boldsymbol{\xi}, \mathbf{x}) = -\|\boldsymbol{\xi} - \mathbf{x}\|_2^2$ or $\mathrm{dot}(\boldsymbol{\xi}, \mathbf{x}) = \boldsymbol{\xi}^\top \mathbf{x}$.

k-**optimal similarity**: similarity on the k dimensions where $\boldsymbol{\xi}$ and $\mathbf{x}$ agree best,

$$\mathrm{sim}^{(k)}(\boldsymbol{\xi}, \mathbf{x}) = \max_{D \subseteq [d],\, |D|=k} \mathrm{sim}(\boldsymbol{\xi}_D, \mathbf{x}_D).$$

Footprint: all of them at once, $\mathrm{ftpt}_{\mathrm{sim}}(\boldsymbol{\xi}, \mathbf{x}) = \left[\mathrm{sim}^{(1)}, \ldots, \mathrm{sim}^{(d)}\right]^\top \in \mathbb{R}^d$.

Cheap to compute (Thm. 3): with per-dimension scores $q_i = \mathrm{sim}(\xi_i, x_i)$ sorted in decreasing order as $\tilde{\mathbf{q}}$,

$$\mathrm{ftpt}_{\mathrm{sim}}(\boldsymbol{\xi}, \mathbf{x}) = \mathbf{U}\tilde{\mathbf{q}} \quad (\mathbf{U} = \text{lower-triangular ones} \Rightarrow \text{cumulative sum}), \qquad O(d \log d).$$

In words

Score every dimension, sort from best to worst, take running totals. The k-th entry = "how similar are we if we only look at our best k dimensions?"

Section 3.2: A 4-dimensional example (presenter's toy)

$\boldsymbol{\xi}_1 = (1,1,1,1)$, $\boldsymbol{\xi}_2 = (0,0,1,1)$, query $\mathbf{x} = (-3,1,1,1) = \boldsymbol{\xi}_1$ with dimension 1 corrupted. Base similarity: $q_i = -(\xi_i - x_i)^2$.

	$\mathbf{q}$	$\tilde{\mathbf{q}}$ (sorted)	$\mathrm{ftpt} = \mathbf{U}\tilde{\mathbf{q}}$	plain distance
$\boldsymbol{\xi}_1$	$(-16,0,0,0)$	$(0,0,0,-16)$	$(0,0,0,-16)$	-16
$\boldsymbol{\xi}_2$	$(-9,-1,0,0)$	$(0,0,-1,-9)$	$(0,0,-1,-10)$	-10

- $\mathbf{w} = (0,0,0,1)$ reads the last entry = ordinary distance $\Rightarrow$ picks $\boldsymbol{\xi}_2$ (wrong).
- $\mathbf{w} = (0,0,1,0)$ reads "best 3 dimensions" $\Rightarrow 0 > -1$, picks $\boldsymbol{\xi}_1$ (right).

Why it works: one badly corrupted dimension dominates the plain distance. Sorting pushes it to the end, and $\mathbf{w}$ learns not to trust the end.

Section 3.3: Adaptive similarity and A-Hop

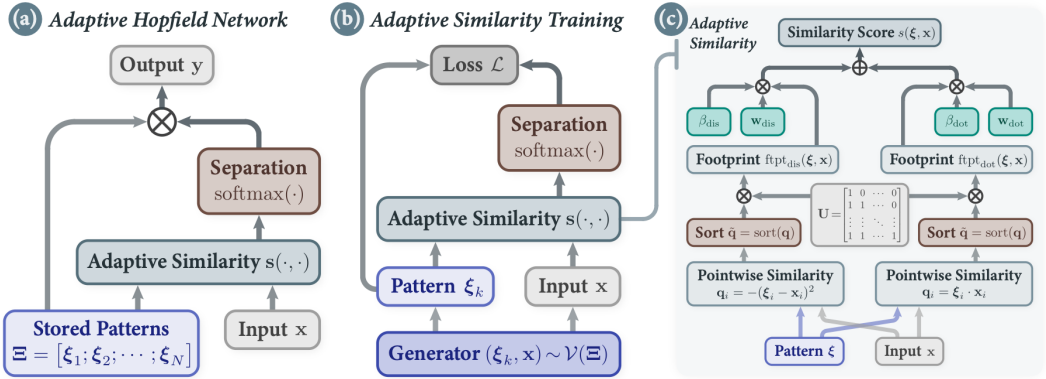
Adaptive similarity = learned weights on the footprint, for two base similarities:

$$s(\boldsymbol{\xi}, \mathbf{x}) = \beta_{\text{dis}}\, \mathbf{w}_{\text{dis}}^{\top} \text{ftpt}_{\text{dis}}(\boldsymbol{\xi}, \mathbf{x}) + \beta_{\text{dot}}\, \mathbf{w}_{\text{dot}}^{\top} \text{ftpt}_{\text{dot}}(\boldsymbol{\xi}, \mathbf{x}), \qquad \mathbf{y} = \boldsymbol{\Xi}\, \text{softmax}\big(s(\boldsymbol{\Xi}, \mathbf{x})\big) \quad \text{(Eq. 4)}$$

What it reduces to

- $\mathbf{w}^{\top}\mathbf{U}\tilde{\mathbf{q}} = \sum_i u_i \tilde{q}_i$ with $u_i = \sum_{j \geq i} w_j$: a learned weighted sum of *sorted* scores. In statistics this is an L-statistic; the trimmed mean is a special case.
- $\mathbf{w} = \mathbf{e}_d$ recovers the base similarity, so A-Hop is at least as expressive as M-Hop.
- Parameters: $2d$ weights + 2 scalars. The same $\mathbf{w}$ is used for every memory.

Section 3.3: Architecture and training



(a) A-Hop = M-Hop with the dot product replaced by the adaptive similarity. **(b)** Training: a generator draws $(\boldsymbol{\xi}_k, \mathbf{x}) \sim \mathcal{V}(\boldsymbol{\Xi})$; the loss rewards putting softmax mass on $\boldsymbol{\xi}_k$. **(c)** Similarity module: two footprints (dis, dot), each sorted, accumulated by $\mathbf{U}$, weighted by $\mathbf{w}$, combined by β.

Section 3.3: Learning

Training objective (Eq. 5): draw pairs from the variant distribution and maximize the softmax probability of the true origin,

$$\mathcal{L}(\Xi, \mathcal{V}) = \mathbb{E}_{(\xi_k, \mathbf{x}) \sim \mathcal{V}(\Xi)} \big[-\log \operatorname{softmax}\big(s(\Xi, \mathbf{x})\big)_k \big].$$

This is cross-entropy for the N-way "which memory?" classification.

The key requirement

Training needs pairs $(\xi_k, \mathbf{x})$ where we **know which memory produced each query**.

- Synthetic retrieval: a generator provides this.
- Tabular / image / MIL experiments: no such labels; Eq. 5 is **dropped** and A-Hop is trained end-to-end on the task loss (App. A.4.4).

"Without assuming the variant type is known a priori" means: no parametric form for $p_{\mathcal{V}}$ is assumed. It does *not* mean unsupervised.

Section 4: Theoretical results

Theorem 1 (optimal correct retrieval). With a "careful design" of s, $\mathcal{T}(\mathbf{x}) = \Xi\,\mathrm{sep}(s(\Xi, \mathbf{x}))$ achieves correct retrieval for every query, for noisy, masked and biased variants.
Argument: use $\mathrm{sep} = \arg\max$ and pick s to be monotone in $\log p_{\mathcal{V}}(\mathbf{x} \mid \boldsymbol{\xi})$ for each variant (Lemmas 2–4).

Theorem 2 (energy). For $s(\boldsymbol{\xi}, \mathbf{x}) = -\|\mathbf{x} - \boldsymbol{\xi}\|^2 + \mathbf{b}^\top(\mathbf{x} - \boldsymbol{\xi})$ and $E(\mathbf{x}) = -\mathrm{lse}(s(\Xi, \mathbf{x}))$, the update decreases E, converges, and $E \geq -\ln N - \|\mathbf{b}\|_2^2/4$. Covers isotropic noisy and biased variants.
Argument: write $E = \|\mathbf{x}\|^2 - \mathrm{lse}(A\mathbf{x} + \mathbf{c})$ (convex minus convex) and apply CCCP.

Practical takeaway. The *family* of learned similarities contains Bayes-optimal retrievers for these three corruptions, and a stable energy exists for a simple member of it.

Section 4: What is proved — and what is not

1. **Thm. 1 is close to a restatement.** If retrieval uses arg max of the log-likelihood, it equals the Bayes rule by definition.
2. **The lemmas do not use the trained model.**
   - Lemmas 2, 4 use the *unsorted* $\mathbf{Uq}$; Lemma 4 adds an extra $-\mathbf{q}^\top\mathbf{q}$ term.
   - Lemma 3 (masked) uses a discrete $\sum_i \delta(\tilde{q}_i)$.
   - The authors state the continuous, sorted $\mathbf{w}^\top\mathbf{U}\tilde{\mathbf{q}}$ is *not* guaranteed optimal (App. A.3.1); they give Monte Carlo error rates for hand-set $\mathbf{w}$ instead.
3. **Thm. 2 uses yet another similarity**, and its two derivations disagree in sign: Eq. 8 gives $\mathbf{x}^{(t+1)} = \Xi\mathbf{p} + \frac{1}{2}\mathbf{b}$, Lemma 6 gives $\Xi\mathbf{p} - \frac{1}{2}\mathbf{b}$.
4. **Smaller issues:** the masked "density" mixes point masses with continuous parts; Dirac δ is used as a 0/1 indicator; $\tilde{\mathbf{q}}$ is "decreasing" in the text but "ascending" in the notation table.

Bottom line

The theory motivates the *idea*. The model that is actually trained is a reasonable heuristic member of the family, justified by experiments rather than theorems.

Section 5: Implementation details

Core computation (my sketch, not the official code)

```
# Xi: (N, d) memories, x: (B, d) queries
q_dis = -(Xi[None] - x[:, None])**2    # (B, N, d)
q_dot =  Xi[None] * x[:, None]         # (B, N, d)
f_dis = q_dis.sort(-1, descending=True)\
            .values.cumsum(-1)         # U q~
f_dot = q_dot.sort(-1, descending=True)\
            .values.cumsum(-1)
logits = b_dis * f_dis @ w_dis \
       + b_dot * f_dot @ w_dot         # (B, N)
y    = logits.softmax(-1) @ Xi         # Eq. 4
loss = F.cross_entropy(logits, origin) # Eq. 5
```

Cost

- Time $O(Nd \log d)$ vs. $O(Nd)$; memory $O(BNd)$ — no $QK^\top$ matmul shortcut.
- Table 14: inference $\approx 1.2\times$ slower with sorting. The training-time row is inconsistent: text says sorted is $2\times$ slower, table shows it faster.

Unclear or buried

- Initialization of $\mathbf{w}$, β (code only).
- Gradients pass through `sort` as a permutation; ties are ignored.
- Tabular model is $\mathrm{MLP}(\mathrm{LayerNorm}(s(\Xi, \mathbf{x})))$ with K-means prototypes and *MSE* loss — the read-out step is not used.

Section 6: Memory retrieval (main evidence)

Setup: random patterns, $d = 64$, $N \in \{2048, 4096\}$; MNIST, $N = 2048$, $d = 784$. Queries corrupted by a mix of masking, noise and bias with intensities $(d_{\text{mask}}, d_{\text{noise}}, d_{\text{bias}})$.

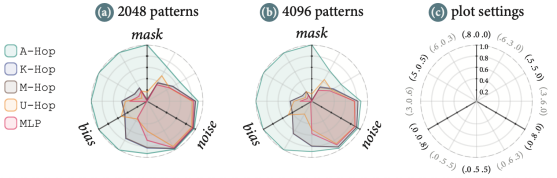


Fig. 3: accuracy over 12 corruption mixes. Baselines collapse on the mask and bias axes.

Dataset	Synthetic				MNIST			
Difficulty	0.4		0.5		0.6		0.7	
Metrics	Accuracy	Error	Accuracy	Error	Accuracy	Error	Accuracy	Error
M-Hop	.520±.02	.176±.01	.195±.03	.300±.01	.875±.01	.013±.00	.661±.02	.068±.00
N-Hop	.260±.04	.417±.02	.059±.01	.554±.01	.540±.03	.143±.01	.176±.02	.347±.01
U-Hop	.487±.03	.295±.02	.177±.02	.764±.02	.764±.02	.064±.01	.526±.02	.164±.01
U_2-Hop	.521±.02	.176±.01	.195±.02	.298±.01	.878±.01	.013±.00	.660±.01	.068±.01
A-Hop	**.724**±.02	**.106**±.01	**.360**±.02	**.227**±.01	**.939**±.01	**.005**±.00	**.849**±.01	**.015**±.01

Table 2: e.g. synthetic, difficulty 0.4: A-Hop .724 vs. M-Hop .520; MNIST, 0.7: .849 vs. .661.

Caveat: M-Hop, N-Hop and U-Hop have no parameters that could learn a bias or ignore masked dimensions. The fairer baseline is missing: a learned per-dimension weighted distance plus offset, *without* sorting.

Section 6: Ablations — where does the gain come from?

Conditions		Synthetic ($d = 0.4$)		Synthetic ($d = 0.5$)	
$\mathbf{q}$ sorted?	use $\mathbf{U}$?	Accuracy	Error	Accuracy	Error
✗	✗	.5172$_{\pm.034}$	.1900$_{\pm.017}$	.2094$_{\pm.022}$	.2658$_{\pm.005}$
✗	✓	.5444$_{\pm.007}$	.1665$_{\pm.007}$	.1888$_{\pm.015}$	.2738$_{\pm.003}$
✓	✗	.6928$_{\pm.034}$	.1173$_{\pm.010}$	.3374$_{\pm.025}$	.2324$_{\pm.006}$
✓	✓	**.7280**$_{\pm.034}$	**.1033**$_{\pm.011}$	**.3634**$_{\pm.040}$	**.2207**$_{\pm.011}$

Table 9: sorting and **U** (difficulty 0.4): neither .517 → both .728.

Conditions		Synthetic ($d = 0.4$)		Synthetic ($d = 0.5$)	
use ftpt_{dis}?	use ftpt_{dot}?	Accuracy	Error	Accuracy	Error
✗	✓	.5926$_{\pm.016}$	.1517$_{\pm.005}$	.2520$_{\pm.027}$	.2515$_{\pm.004}$
✓	✗	.6458$_{\pm.042}$	.1317$_{\pm.011}$	.3286$_{\pm.023}$	.2326$_{\pm.007}$
✓	✓	**.7242**$_{\pm.016}$	**.1056**$_{\pm.007}$	**.3600**$_{\pm.024}$	**.2272**$_{\pm.005}$

Table 11: dot only .593, dis only .646, both .724.

Reading the ablations

- **Sorting is the main ingredient**: removing it costs far more than removing **U**.
- **The loss is not the reason**: U2-Hop (U-Hop kernel trained with the same Eq. 5) ≈ M-Hop in Table 2.
- **Sample-efficient**: 512 training pairs ≈ infinite pairs for $N = 2048$ (Table 12).

This is the most convincing chain of evidence in the paper.

Section 6: Downstream tasks

Model	Adult	Bank	Vaccine	Purchase	Heart
M-Hop	.8080±.001	.9085±.003	.7975±.001	.8822±.001	.6325±.002
U2-Hop	.8172±.003	.9092±.002	.7971±.003	.8825±.002	.6473±.002
A-Hop	**.8634**±.002	**.9139**±.002	**.8042**±.002	**.9007**±.001	**.7315**±.002
Extra Trees	.8595±.004	.9098±.003	.7932±.002	.8916±.002	.7175±.003
Random Forest	.8592±.002	.9132±.003	.7918±.003	.9002±.001	.7254±.002
AdaBoost	.8597±.003	.9094±.001	.8011±.002	.8865±.001	.7294±.001
XGBoost	.8640±.002	.9152±.003	.8034±.002	.9032±.003	.7370±.003

Table 3: tabular classification.

Image Classification				*Multiple Instance Learning*				
Dataset	CIFAR10	CIFAR100	Tiny ImageNet	Dataset	Tiger	Fox	Elephant	UCSB
M-Hop	.5123±.003	.2464±.003	.1095±.002	M-Hop	.8924±.005	.6327±.013	.9344±.009	.8815±.022
U-Hop	.5489±.002	.2877±.002	.1164±.002	S-Hop	.8923±.006	.6433±.015	.9365±.002	.8794±.024
A-Hop	**.5637**±.003	**.2904**±.002	**.1213**±.002	A-Hop	**.9030**±.007	**.6753**±.013	**.9451**±.004	**.8935**±.022

Table 4: image classification and MIL.

What holds up

- Consistently best among Hopfield variants.
- Largest gain on *Heart* (cardiovascular disease, 70k patients): .7315 vs. .6473 (U2-Hop).

My doubts

- XGBoost still wins 4 of 5 tabular datasets (Heart: .7370).
- Random Forest / AdaBoost are called "deep learning baselines".
- MIL gains lie within one std. (UCSB .894±.022 vs. .882±.022).
- CIFAR-10 accuracy ≈56% for all models — a weak setting.
- No fixed random seed (Reproducibility statement).

Section 7: Recommendations

Is it compelling?

- **Idea**: yes. "Similarity should be the log-likelihood of the corruption process" is a useful lens for any attention or retrieval module.
- **Method**: plausibly useful when some input dimensions are unreliable. Cheap ($2d + 2$ parameters).
- **Theory**: does not cover the trained model. Treat the theorems as motivation.

When I would try it: retrieval where (i) query and memory live in the same space, (ii) some dimensions are corrupted or missing, and (iii) we can generate (query, origin) pairs.

Ideas for our work

- In patient-memory retrieval, the query is *not* a corrupted copy of a memory item, and we have no "which item should be retrieved" labels — only the downstream outcome. How should "correct retrieval" be defined there?
- Synthetic probes (planted signal in a known memory slot) are one way to *construct* a variant distribution and make Eq. 5 usable.
- Cheap ablation: replace cosine in top-L retrieval with a sorted, learned-weight similarity.

Discussion

1. The same **w** is shared by every memory and every query. When would per-memory or query-dependent weights be necessary? (App. A.3.2 raises this.)
2. Sorting throws away *which* dimensions agree. For EHR features, is that a feature (robustness) or a bug (interpretability)?
3. Without origin labels, A-Hop is just a learnable similarity trained by the task loss. What, then, is left of the "correct retrieval" framing?
4. Would a per-dimension weighted distance with offset (no sorting) close most of the gap in Table 2?
5. Is "retrieval = Bayes classification" a good lens for attention over a patient's history, where many memories can be relevant at once?

Backup: Lemma 2 — weights for the anisotropic noisy variant

Right-hand side of Eq. 1 for $\mathbf{x} - \boldsymbol{\xi} \sim \mathcal{N}(0, \mathrm{diag}(\boldsymbol{\sigma}))$:

$$\arg\max_{\boldsymbol{\xi}'} p_{\mathcal{V}}(\boldsymbol{\xi}' \mid \mathbf{x}) = \arg\max_{\boldsymbol{\xi}'} \Big(- \sum_i (\xi_i' - x_i)^2 / \sigma_i \Big).$$

With the *unsorted* footprint $\mathbf{Uq}$, $q_i = (\xi_i - x_i)^2$, and $\mathbf{u}^\top = \mathbf{w}^\top \mathbf{U}$, choose $u_i = -1/\sigma_i$:

$$w_1 = -\sigma_1^{-1}, \qquad w_i = \sigma_{i-1}^{-1} - \sigma_i^{-1} \ \ (2 \le i \le d).$$

Since $\mathbf{U}$ is invertible, any per-dimension weighting is reachable. **But** this needs the dimension order to be fixed — which sorting destroys. That is why the sorted model is not covered.

Backup: Theorem 2 via CCCP

For $s(\boldsymbol{\xi}_i, \mathbf{x}) = -\|\mathbf{x} - \boldsymbol{\xi}_i\|^2 + \mathbf{b}^\top(\mathbf{x} - \boldsymbol{\xi}_i)$:

$$E(\mathbf{x}) = \underbrace{\|\mathbf{x}\|^2}_{g \text{ convex}} \underbrace{- \operatorname{lse}(A\mathbf{x} + \mathbf{c})}_{h \text{ convex}}, \qquad A_i^\top = 2\boldsymbol{\xi}_i + \mathbf{b}, \;\; c_i = \boldsymbol{\xi}_i^\top \mathbf{b} - \|\boldsymbol{\xi}_i\|^2.$$

CCCP step: minimize $g(\mathbf{x}) - \nabla h(\mathbf{x}^{(t)})^\top \mathbf{x}$, giving $\mathbf{x}^{(t+1)} = \frac{1}{2} A^\top \mathbf{p}(\mathbf{x}^{(t)}) = \boldsymbol{\Xi}\mathbf{p} + \frac{1}{2}\mathbf{b}$ (the paper writes $-\frac{1}{2}\mathbf{b}$ here).

Monotone decrease: $E(\mathbf{x}^{(t+1)}) \le U_t(\mathbf{x}^{(t+1)}) \le U_t(\mathbf{x}^{(t)}) = E(\mathbf{x}^{(t)})$.
Lower bound: $-r^2 + \|\mathbf{b}\| r \le \|\mathbf{b}\|^2/4$ for every $r = \|\mathbf{x} - \boldsymbol{\xi}_i\|$, so $E \ge -\ln N - \|\mathbf{b}\|^2/4$.

Backup: masked variant with hand-set weights (App. A.3.1)

Assume masked dimensions differ from $\boldsymbol{\xi}$ much more than noisy ones. If exactly m dimensions are masked, reading entry $d - m$ of the footprint ignores them. Averaging over $m \sim \mathrm{Binomial}(d, p_{\text{masked}})$:

$$w_i = \binom{d}{i} p_{\text{masked}}^{d-i} (1 - p_{\text{masked}})^i.$$

Monte Carlo error with this $\mathbf{w}$ ($d = 64$, patterns in $[-1, 1]^d$):

p_{masked}	0.1	0.25	0.5	0.75
$\hat{p}_{\text{err}}$	6.8×10^{-14}	1.6×10^{-9}	1.2×10^{-6}	10^{-3}

These are for *hand-set* weights and pure masking — not for weights learned under mixed corruption.